SOFTWARE ENGINEER · FLORIANÓPOLIS, BRAZIL

Danilo *Silva*

I build AI developer tools and the systems around them: the Rust engine, the interface, the benchmarks that decide what to trust, and the signed installer at the end.

danilosilvadev@gmail.com · linkedin.com/in/danilo-silva-81439369 · github.com/danilosilvadev · antiloki.com

01 ABOUT

Ten years of front-end engineering at banks and regulated-data companies. Now the whole stack, on my own: a Rust engine, a desktop app, a model router driven by benchmarks, and the release pipeline behind them. I direct AI agents under rules I wrote. Tests first, review gates, and nothing claimed that was not measured.

YEARS IN FRONT-END	HTTP ROUTES IN THE ENGINE	AUTOMATED TESTS	SIGNED RELEASES SHIPPED
10+	328	10,000+	11

02 WHAT I BUILD

Four things, done to the end.

01 — AI SYSTEMS & EVALUATION

Decide from evidence, then check the result.

- A router that sends each coding job to the model that benchmarks best for it, with quota-aware fallback when a subscription runs out.
- Confidence that is itself measured: does "sure" turn out right? Calibrated against real outcomes.
- A deterministic layer of small gauges that answer from facts they can point to, and escalate to a model only when they cannot.
- Benchmark and judge harnesses with their pitfalls written down, and claims retracted when a re-run did not reproduce them.

02 — SYSTEMS ENGINEERING

Rust that holds up on someone else's machine.

- An async engine (axum, SQLite) that runs five different command-line AI tools as live sessions, on Windows and Linux.
- Process and terminal orchestration, streaming over server-sent events, deadlock-safe database access.
- A TypeScript engine ported to Rust under conformance fixtures and byte-parity tests, so behaviour did not drift.
- About 360,000 lines of Rust and more than 5,800 engine tests.

03 — FRONT END & PRODUCT DESIGN

Interfaces that stay fast and make sense.

- React and TypeScript, about 100,000 lines and 3,400 tests; runtime-switchable themes on one set of design tokens.
- A conflict-safe code editor, a Linear-style issue tracker, live-markdown notes, a code-analysis cockpit.
- A shared stream hub so a dozen live panes stay under the browser's per-host connection cap.
- Onboarding that checks which AI tools you have and leads you to the missing ones.

04 — SHIPPING & OPERATIONS

From commit to a customer's desktop.

- Tauri desktop packaging; the Windows build cross-compiled from Linux; signed installers and an auto-updater.
- Licensing with a trial, provider-validated keys and offline signed serials, designed so a paying user is never locked out.
- Payments integration, landing site, technical paper, newsletter, privacy and terms, release notes.
- Clean-machine testing that found, and fixed, the Windows-only failures before customers did.

03 CASE STUDY

antiloki.

One coding agent from the subscriptions you already pay for.

STACK

Tauri · Rust (axum, SQLite) · React · TypeScript

PLATFORMS

Windows and Linux, signed installers, auto-update

SEATS

Claude Code, Codex, Cursor, Gemini, Grok

PRINCIPLES

Local-first. No telemetry. No server of ours.

Source is private. The technical paper, the release notes and a download are at antiloki.com. Built with AI agents as collaborators: I own the architecture, the acceptance tests and every release decision.

04 ALSO BUILT

RESEARCH — IMIR

Memory for AI with no model at inference.

A deterministic language-and-memory substrate in Rust: a person's conversations compiled into six separate memory worlds in one SQLite file, every answer traceable to a rule and a sentence. Evaluated on the LoCoMo benchmark with the limits stated openly, and written up as a paper.

RUST · SQLITE · EVALUATION · RESEARCH

GO-TO-MARKET — MUNINN

The outreach engine behind the launch.

Waitlist, leads, messaging, suppression lists and a job queue on Supabase (Postgres and Edge Functions), a NestJS API and an operator console in React. Built so sending is governed: consent first, unsubscribes always honoured.

TYPESCRIPT · NESTJS · REACT · POSTGRES

05 METHOD

How I work.

01 **Measure before claiming.**
A claim without a measurement is not a claim. I keep the numbers that argue against me: a cost advantage I had published was removed when a re-run no longer reproduced it.

02 **Deterministic first.**
Rules decide what rules can decide, and say what they can point to. A model is for the questions about meaning, with a measured miss rate.

03 **Never lock out a legitimate user.**
Protection is designed around the customer who pays and reinstalls on a new laptop. Offline grace, honest fallbacks, no secrets in the binary.

04 **Small, tested steps, written rules.**
One change at a time, a test before the claim, builds capped by memory, nothing pushed or released without approval. AI agents work inside rules I write down, and the rules survive the session.

05 **Ship, then listen.**
Every release is checked as a stranger would download it, then tested on a clean machine. What breaks there is the roadmap.

06 PATH

Ten years, one direction: systems that are easy to change and hard to break.

2025 — NOW

Founder & lead engineer

antiloki · Intriguing Possibilities

- Designed and built the product end to end: desktop app, Rust engine, router, release pipeline.
- Eleven signed releases; benchmark-driven routing; 10,000+ automated tests.
- Go-to-market: landing, technical paper, newsletter, outreach engine.

2022 — 2025

React architect, senior

Arrow Global · Portugal

- Refactored a legacy React/TypeScript code base to SOLID with custom hooks and context; components from hundreds of lines to under 60.
- Built a self-generating multi-step form (useReducer, Context) that replaced Excel tables for fund investments.
- MirageJS mock API, commit linting, an in-house CSS framework.

2021 — 2022

Senior frontend developer

CIBC FirstCaribbean International Bank

- Started a credit-card selection application from scratch; led the technology and architecture decisions.
- React, Redux Toolkit, TypeScript, Cypress; extensive custom hooks cut component size by up to 40%.

2019 — 2021

Senior frontend developer

K2 Partnering Solutions

- Proved a functional-programming rewrite with a concept branch (200 lines became 30); code size fell about 35% on a product earning over €5 million a year.
- Vue learning platform; helped reach 80% coverage; sprint demos to clients in English.

2019

Senior frontend developer

Dogma Data Privacy

- Alone, from scratch: a GDPR permissions and cookie dashboard with 89% coverage, Jest and Cypress, Storybook.

2016 — 2019

Frontend developer

BlueFoot · Trackmob · FaçoDesign

- E-commerce component library and data-cleaning middleware at 80%+ coverage, leading a small team; an NPO donations dashboard with internationalisation.

EDUCATION — BSC COMPUTER SCIENCE, UNIVERSITY OF THE PEOPLE (2018 - 2022)
LANGUAGES — PORTUGUESE (NATIVE), ENGLISH (PROFESSIONAL)

07 TOOLBOX

Rust · TypeScript · React · axum / tokio · SQLite · Tauri · Vite / Vitest · CodeMirror · SSE streaming · Model routing · Evaluation harnesses · Calibration · Agent orchestration · Guardrails · Prompt packs · Cross-compilation · Code signing